

Procesador RISC-V Pipelined con Extensión C

Pipeline, Hazard Unit e Instrucciones Comprimidas

Ildefonso Santos
Steve Andy

Paucar Barrios
Miguel Luis

Rosillo Rodríguez
Marcelo Mateo

UTEC — Arquitectura de Computadoras

Julio 2026

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit
- 3 Extensión C: instrucciones comprimidas
- 4 Resultados: algoritmo y comparativa
- 5 Conclusiones

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit
- 3 Extensión C: instrucciones comprimidas
- 4 Resultados: algoritmo y comparativa
- 5 Conclusiones

- **E1 (5 pts):** pipeline base de 5 etapas + Hazard Unit (forward/stall/flush) + ISA RV32I (24 instrucciones).

Ciclo por fase

implementar → programa de prueba → testbench → waveform → siguiente fase. Este patrón se ve en detalle en los próximos dos capítulos.

Metodología: incremental, por entregas

- **E1 (5 pts):** pipeline base de 5 etapas + Hazard Unit (forward/stall/flush) + ISA RV32I (24 instrucciones).
- **E2 (4 pts):** extensión C, parte 1 — 11 instrucciones comprimidas de ALU/registro-inmediato.

Ciclo por fase

implementar → programa de prueba → testbench → waveform → siguiente fase. Este patrón se ve en detalle en los próximos dos capítulos.

Metodología: incremental, por entregas

- **E1 (5 pts):** pipeline base de 5 etapas + Hazard Unit (forward/stall/flush) + ISA RV32I (24 instrucciones).
- **E2 (4 pts):** extensión C, parte 1 — 11 instrucciones comprimidas de ALU/registro-inmediato.
- **E3 (3 pts):** extensión C, parte 2 — 10 instrucciones de memoria y control de flujo.

Ciclo por fase

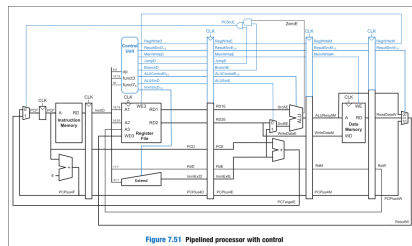
implementar → programa de prueba → testbench → waveform → siguiente fase. Este patrón se ve en detalle en los próximos dos capítulos.

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit**
- 3 Extensión C: instrucciones comprimidas
- 4 Resultados: algoritmo y comparativa
- 5 Conclusiones

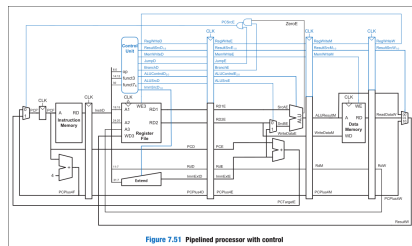
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones



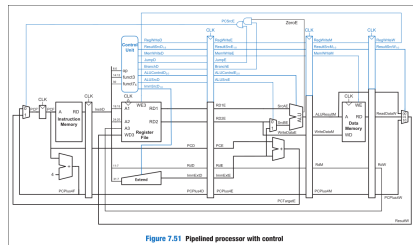
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend



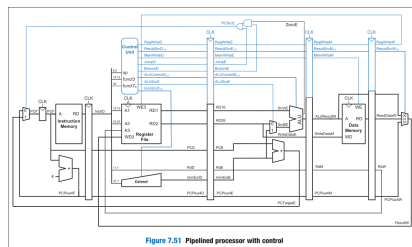
El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend
- **Execute:** ALU, cálculo de saltos



El pipeline base: 5 etapas

- **Fetch:** PC, memoria de instrucciones
- **Decode:** banco de registros, extend
- **Execute:** ALU, cálculo de saltos
- **Memory:** memoria de datos
- **Writeback:** escritura al banco de registros



El precio de segmentar

Varias instrucciones en vuelo a la vez $\Rightarrow$ los datos y saltos no siempre están listos a tiempo.

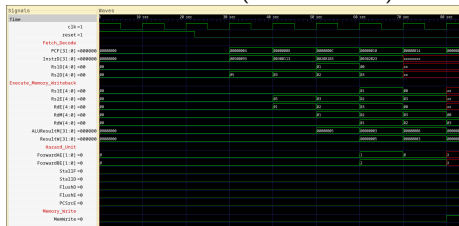
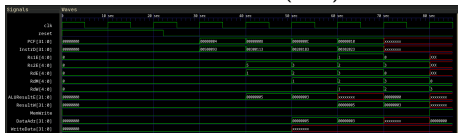
- **De datos (RAW):** una instrucción usa un resultado que otra anterior aún no escribió. Caso crítico: *load-use*.
- **De control:** los saltos se resuelven en Execute, pero ya se captaron instrucciones secuenciales de más.
- **Estructurales:** evitados por diseño (imem/dmem separadas, regfile lee y escribe en el mismo ciclo).

Forward: adelantar el dato

El programa quiere usar un resultado apenas calculado en la instrucción siguiente; sin *forwarding* lee el valor viejo del banco de registros.

Con Hazard Unit (ForwardAE/BE)

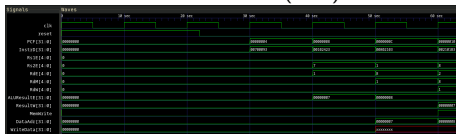
Sin Hazard Unit (falla)



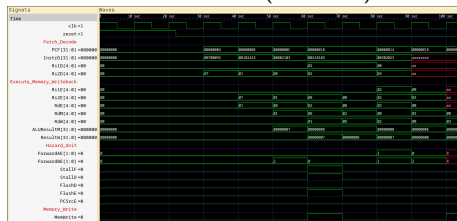
Stall: el caso *load-use*

El programa quiere usar un valor que recién se está leyendo de memoria; no se puede resolver solo con *forwarding* — hay que detener el pipeline un ciclo.

Sin Hazard Unit (falla)



Con Hazard Unit (StallF/D)

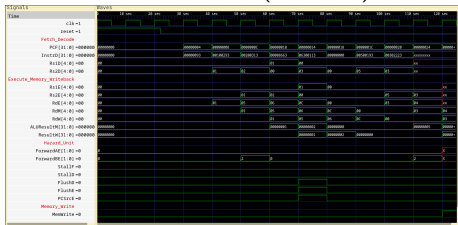
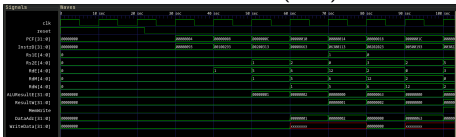


Flush: descartar tras un salto

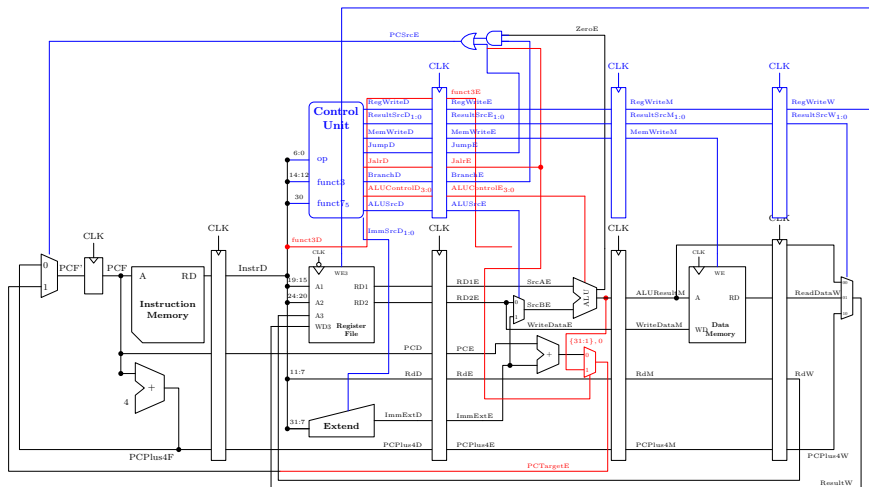
El programa tiene un salto/branch; las instrucciones ya cargadas en el pipeline detrás del salto son incorrectas y hay que descartarlas.

Con Hazard Unit (FlushD/E)

Sin Hazard Unit (falla)



Datapath final: implementación de instrucciones



En rojo: adiciones propias sobre el pipeline base — Jalr, funct3E, ALUControl1 de 4 bits, mux de destino jalr.

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit
- 3 Extensión C: instrucciones comprimidas**
- 4 Resultados: algoritmo y comparativa
- 5 Conclusiones

¿Qué es RVC?

- Instrucciones de **16 bits** que coexisten con las de 32 bits.
- Reducen el tamaño del código **25–30 %** sin perder expresividad.
- Cada comprimida tiene **exactamente un** equivalente de 32 bits.

Estrategia: descomprimir en Fetch

Expandir los 16 bits a RV32I **antes** del registro IF/ID. Decode, Execute, Memory, Writeback, controller, hazardunit **no se modifican**.

Validación incremental por familia

Cada familia se implementó, probó y validó **antes** de pasar a la siguiente — no se validó todo junto al final.

Entrega	Familia (formato)	Programa / resultado
E2	CI: c.addi, c.lui, c.slli	caddi, cir ✓
E2	CR: c.add	cir ✓
E2	CA: c.sub, c.xor, c.or, c.and	ca ✓
E2	CB: c.srli, c.srai, c.andi	cb ✓
E3	CL/CS: c.lw, c.sw	clsw ✓
E3	CI/CSS: c.lwsp, c.swsp	clspsp ✓
E3	CB: c.beqz, c.bnez	cbz ✓
E3	CJ: c.j, c.jal	cj ✓
E3	CR: c.jr, c.jalr	cjr ✓

Cambios en el datapath

- 1 **Ventana deslizante** en imem: una comprimida puede dejar al PC desalineado a palabra.

```
assign rd = a[1] ? {w1[15:0], w0[31:16]} : w0;
```

- 2 **Incremento variable**: $PC += \text{compressed} ? 2 : 4$.
- 3 **Módulo** decompressor.v: combinacional, produce la instrucción expandida antes del IF/ID.

Costo en ciclos

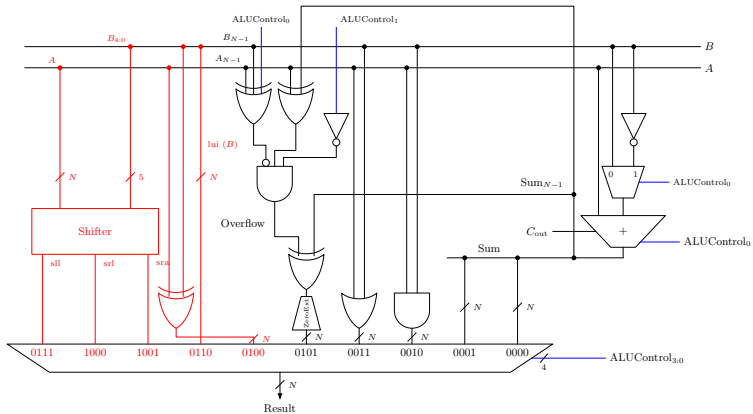
Cero. Todo es combinacional dentro de la etapa Fetch.

Módulos agregados / afectados

Módulo	Rol
<code>decompressor.v</code>	Nuevo. Expande 16→32 bits (combinacional)
<code>imem.v</code>	Ventana deslizante para PC desalineado
<code>datapath.v</code>	PCPlusInc, instancia del decompressor
<code>alu.v</code>	+5 operaciones: xor, lui, sll, srl, sra

Sin cambios: `controller`, `extend`, `regfile`, `hazardunit`, `dmem`

ALU modificada



Rojo: xor, lui, y el *shifter* (sll/srl/sra). Mux de salida creció de 5 a 10 entradas; ALUControl de 3 a 4 bits.

Las 21 instrucciones comprimidas

Parte 1 (E2) — 11 instr.

- CL: `c.addi`, `c.lui`, `c.slli`
- CR: `c.add`
- CA: `c.sub`, `c.xor`, `c.or`, `c.and`
- CB: `c.srli`, `c.srai`, `c.andi`

Parte 2 (E3) — 10 instr.

- CL/CS: `c.lw`, `c.sw`
- CI/CSS: `c.lwsp`, `c.swsp`
- CB: `c.beqz`, `c.bnez`
- CJ: `c.j`, `c.jal`
- CR: `c.jr`, `c.jalr`

Ejemplo: c.lw / c.sw

Registros restringidos x8-x15, *offset* de 7 bits **barajado** (*scrambled*):

```
wire [6:0] offlw = {c[5], c[12:10], c[6], 2'b00};

5'b00_010: instr = {5'b0, offlw, rs1p, 3'b010,
                    rs2p, 7'b0000011};           // c.lw
5'b00_110: instr = {5'b0, offlw[6:5], rs2p, rs1p,
                    3'b010, offlw[4:2], 2'b00,
                    7'b0100011};                 // c.sw
```

Ejemplo: desambiguación c.jr/c.jalr/c.add

Comparten {op,funct3}={10,100} (formato CR) junto con c.mv (subproducto de la desambiguación, no requerido). Se distinguen con c[12] y si rs2 es cero:

```
if (!c[12] && rs2 == 5'b0)
    instr = {12'b0, rd, 3'b000, 5'b00000, 7'b1100111}; // c.jr
else if (!c[12])
    instr = {7'b0, rs2, 5'b00000, 3'b000, rd, 7'b0110011}; // c.mv
else if (rs2 == 5'b0)
    instr = {12'b0, rd, 3'b000, 5'b00001, 7'b1100111}; // c.jalr
else
    instr = {7'b0000000, rs2, rd, 3'b000, rd, 7'b0110011}; // c.add
```

c[12]	rs2=0?	Instrucción	Expande a
0	sí	c.jr	jalr x0, rs1, 0
1	sí	c.jalr	jalr x1, rs1, 0
1	no	c.add	add rd, rd, rs2

Encoding de las 21 instrucciones

Instr.	op	f3	Fmt
c.addi	01	000	CI
c.jal	01	001	CJ
c.lui	01	011	CI
c.srli	01	100	CB
c.srai	01	100	CB
c.andi	01	100	CB
c.sub	01	100	CA
c.xor	01	100	CA
c.or	01	100	CA
c.and	01	100	CA

Instr.	op	f3	Fmt
c.j	01	101	CJ
c.beqz	01	110	CB
c.bnez	01	111	CB
c.slli	10	000	CI
c.lwsp	10	010	CI
c.jr	10	100	CR
c.jalr	10	100	CR
c.add	10	100	CR
c.swsp	10	110	CSS
c.lw	00	010	CL
c.sw	00	110	CS

$op = c[1:0], funct3 = c[15:13]$

- **Subset de RVC no cubierto:** `c.nop`, `c.li`, `c.addi16sp`, `c.addi4spn`, `c.ebreak` — el foco fueron las 21 instrucciones de E2+E3. Tampoco hay punto flotante comprimido ni RV64C (fuera de alcance de una base RV32I).

Nota aparte

`c.mv` sí está implementado (subproducto de la desambiguación del cuadrante 2), aunque no se presenta como instrucción propia porque no fue pedida explícitamente.

Limitaciones

- **Subset de RVC no cubierto:** `c.nop`, `c.li`, `c.addi16sp`, `c.addi4spn`, `c.ebreak` — el foco fueron las 21 instrucciones de E2+E3. Tampoco hay punto flotante comprimido ni RV64C (fuera de alcance de una base RV32I).
- **Registros restringidos** (limitación del estándar RVC, no del diseño propio): CA, CB, CL y CS solo direccionan x8-x15 — un compilador real tendría que priorizar esos registros para aprovechar la compresión al máximo.

Nota aparte

`c.mv` sí está implementado (subproducto de la desambiguación del cuadrante 2), aunque no se presenta como instrucción propia porque no fue pedida explícitamente.

- **Subset de RVC no cubierto:** `c.nop`, `c.li`, `c.addi16sp`, `c.addi4spn`, `c.ebreak` — el foco fueron las 21 instrucciones de E2+E3. Tampoco hay punto flotante comprimido ni RV64C (fuera de alcance de una base RV32I).
- **Registros restringidos** (limitación del estándar RVC, no del diseño propio): CA, CB, CL y CS solo direccionan x8-x15 — un compilador real tendría que priorizar esos registros para aprovechar la compresión al máximo.
- **Sin detección de encoding ilegal:** un patrón de 16 bits que no corresponda a ninguna instrucción soportada no genera trap — el comportamiento queda indefinido.

Nota aparte

`c.mv` sí está implementado (subproducto de la desambiguación del cuadrante 2), aunque no se presenta como instrucción propia porque no fue pedida explícitamente.

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit
- 3 Extensión C: instrucciones comprimidas
- 4 Resultados: algoritmo y comparativa**
- 5 Conclusiones

Programa algoritmo: búsqueda binaria

Busca 18 en [3,7,12,18,25]; dispara un *load-use hazard* real (c.lw x12 → beq x12, ...), conectando directo con la Hazard Unit.

```
// setup: Mem[32..48]=[3,7,12,18,25]
addi x14,x0,32    // base
addi x8,x0,0      // lo = 0
addi x9,x0,4      // hi = 4
addi x13,x0,18    // target
```

```
LOOP:
    blt    x9,x8,NOTFOUND
    addi   x10,x8,0
    c.add  x10,x9      // mid=lo+hi
    c.srli x10,1      // mid >>= 1
    addi   x11,x10,0
    c.slli x11,2      // addr=mid*4
    c.add  x11,x14     // addr+=base
    c.lw   x12,0(x11)  // value
    beq    x12,x13,FOUND
    blt    x12,x13,GO_RIGHT
```

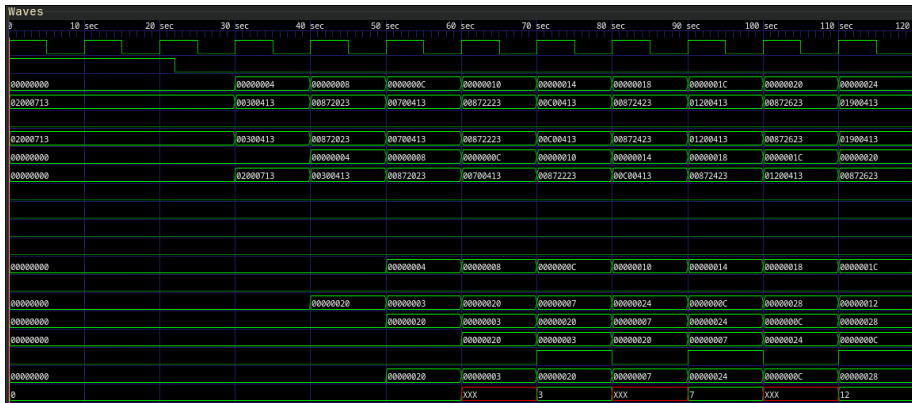
```
GO_LEFT:
    addi   x9,x10,0
    c.addi x9,-1      // hi=mid-1
    c.j    LOOP
```

```
GO_RIGHT:
    addi   x8,x10,0
    c.addi x8,1      // lo=mid+1
    c.j    LOOP
```

```
FOUND:
    sw     x10,20(x14) // indice
    c.j    END
```

```
NOTFOUND:
    addi   x10,x0,-1
    sw     x10,20(x14)
    END:
```


Explicación de resultados (1/5): setup



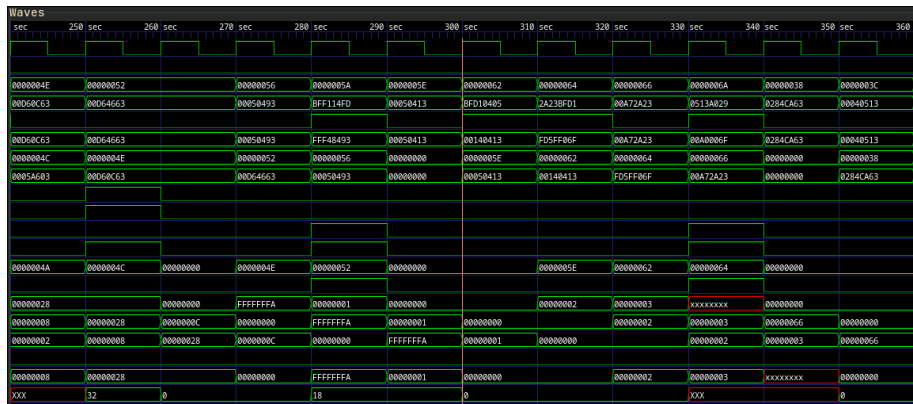
El arreglo [3,7,12,18,25] se escribe en memoria: sw consecutivos hacia Mem[32..48].

Explicación de resultados (2/5): fin de setup, inicio del loop



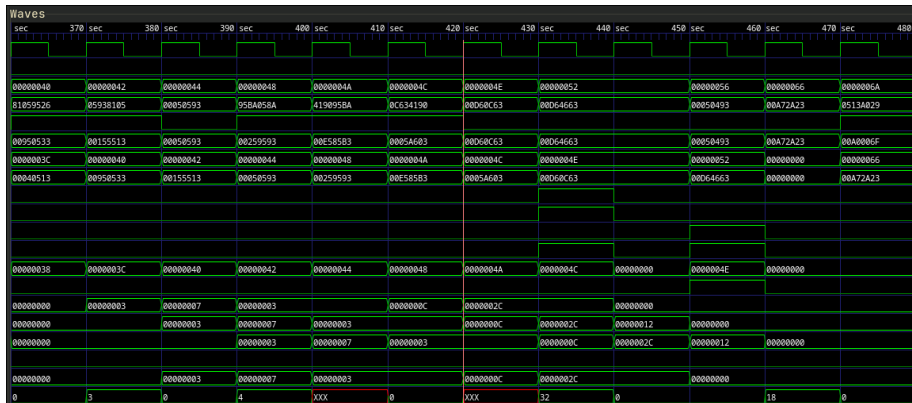
Últimos elementos del arreglo escritos; se inicializan lo, hi, target y el puntero de log — entra al LOOP.

Explicación de resultados (3/5): iteración 1



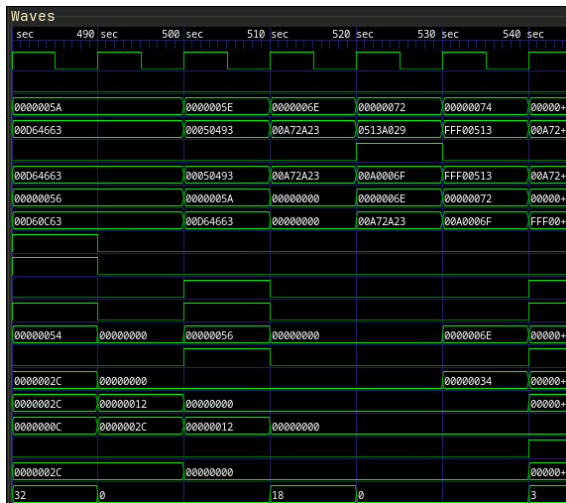
Cálculo de mid, lectura de arr[mid] con c.lw y comparación inmediata en beq — el load-use hazard real, resuelto por la misma Hazard Unit del Capítulo 2.

Explicación de resultados (4/5): iteración 2



Segunda vuelta del loop: nuevo mid, nueva lectura de memoria, nueva comparación — el mismo patrón de la iteración 1, ahora con lo/hi actualizados.

Explicación de resultados (5/5): encontrado



`arr[mid]==target` → encontrado. `Mem[52]=3`: el índice correcto de 18 en `[3,7,12,18,25]`.

Comparativa: *performance*

Hecho medido

RV32I puro y RVC: **545 ciclos en ambas** versiones (mismo algoritmo, mismo resultado).

Razón

El descompresor es combinacional y vive **antes** del registro IF/ID — no agrega etapas ni ciclos. Ninguna instrucción cuesta más por estar comprimida.

Execution Time

$$\text{Execution Time} = \# \text{instrucciones} \times \text{CPI} \times T_c$$

Los tres términos son iguales en ambas versiones: mismo $\#$ instrucciones, mismo CPI, mismo T_c .

Comparativa: tamaño

De 37 instrucciones totales (mismo algoritmo en ambas versiones):

Versión	Instr. 32b	Instr. 16b	Bytes
RV32I puro	37	0	$37 \times 4 = 148$
Con RVC	25	12	$25 \times 4 + 12 \times 2 = 124$

12 de 37 instrucciones comprimidas (32 %) → 24 bytes ahorrados (16 % menos código), con cero costo en ciclos.

Contenido

- 1 Proceso de construcción
- 2 Pipeline y Hazard Unit
- 3 Extensión C: instrucciones comprimidas
- 4 Resultados: algoritmo y comparativa
- 5 Conclusiones

- Pipeline de 5 etapas completo (RV32I, 24 instrucciones) + Hazard Unit funcional, validada con y sin ella (falla sin, funciona con).

- Pipeline de 5 etapas completo (RV32I, 24 instrucciones) + Hazard Unit funcional, validada con y sin ella (falla sin, funciona con).
- Extensión RVC (21 instrucciones, E2+E3) integrada **sin modificar** pipeline/control/hazard unit — todo resuelto en un descompresor combinacional antes de IF/ID.

- Pipeline de 5 etapas completo (RV32I, 24 instrucciones) + Hazard Unit funcional, validada con y sin ella (falla sin, funciona con).
- Extensión RVC (21 instrucciones, E2+E3) integrada **sin modificar** pipeline/control/hazard unit — todo resuelto en un descompresor combinacional antes de IF/ID.
- Validación incremental por fases, regresión completa en verde (26/26 tests).

- Pipeline de 5 etapas completo (RV32I, 24 instrucciones) + Hazard Unit funcional, validada con y sin ella (falla sin, funciona con).
- Extensión RVC (21 instrucciones, E2+E3) integrada **sin modificar** pipeline/control/hazard unit — todo resuelto en un descompresor combinacional antes de IF/ID.
- Validación incremental por fases, regresión completa en verde (26/26 tests).
- bsearch demuestra todo integrado a la vez: pipeline + hazard unit + RVC funcionando juntos, con 16 % menos código y cero costo en ciclos.

- **Timing del regfile (E1):** con dependencia a distancia 3 (WB y Decode en el mismo ciclo), el regfile genérico (posedge) no hacía visible el dato a tiempo. Corregido con **write-first en negedge** (Harris & Harris, Sec. 7.5.1) — puente hacia el forwarding.

- **Timing del regfile (E1):** con dependencia a distancia 3 (WB y Decode en el mismo ciclo), el regfile genérico (posedge) no hacía visible el dato a tiempo. Corregido con **write-first en negedge** (Harris & Harris, Sec. 7.5.1) — puente hacia el forwarding.
- **Bug real en `c.1w` (E3):** el destino vive en `c[4:2]`, no en `c[9:7]` como sugería la analogía con CB. Depurado aislando el `decompressor.v` del resto del pipeline.

- **Timing del regfile (E1):** con dependencia a distancia 3 (WB y Decode en el mismo ciclo), el regfile genérico (posedge) no hacía visible el dato a tiempo. Corregido con **write-first en negedge** (Harris & Harris, Sec. 7.5.1) — puente hacia el forwarding.
- **Bug real en `c.1w` (E3):** el destino vive en `c[4:2]`, no en `c[9:7]` como sugería la analogía con CB. Depurado aislando el `decompressor.v` del resto del pipeline.
- **Volumen de archivos:** más de 25 pares `.mem/testbench` — mantenerlos sincronizados fue un desafío de organización tanto como de diseño.

- Ampliar el subset de RVC (`c.li`, `c.addi16sp`, `c.addi4spn`) y agregar detección de encoding ilegal (trap), hoy ausente.

- Ampliar el subset de RVC (`c.li`, `c.addi16sp`, `c.addi4spn`) y agregar detección de encoding ilegal (`trap`), hoy ausente.
- **Predicción de branches:** cada salto tomado paga el costo de *flush* (visible en el waveform de `bsearch`); una predicción estática simple (*backward=taken*) acercaría el CPI real al ideal de 1.

Gracias

